you'll need in production, including secrets management, ConfigMaps, and Secrets.

Summary

- The Spring `Environment` abstraction provides a unified interface for accessing properties and profiles.
- Properties are key/value pairs used to store configuration.
- Profiles are logical groups of beans registered only when a specific profile is active.
- Spring Boot collects properties from different sources according to precedence rules. From the highest to the lowest precedence, properties can be defined in command-line arguments, JVM system variables, OS environment variables, profile-specific property files, and generic property files.
- Spring beans can access properties from the `Environment` object by injecting the value with the `@Value` annotation, or from a bean mapped to a set of properties with the `@ConfigurationProperties` annotation.
- The active profiles can be defined with the `spring.profiles.active` property.
- The `@Profile` annotation marks beans or configuration classes to be considered only when the specified profile is active.
- Properties, as managed in Spring Boot, provide externalized configuration as defined by the 15-Factor methodology, but that's not enough.